

2026红岩网校工作站AI开发与应用部 Agent 方向学习指南

一、学习和考核说明

本学期 Agent 方向共有 8 个考核任务需要大家进行自学和完成。在本学期结束后，我们将会根据大家的考核任务完成质量以及数量，筛选进入最后的暑假考核的同学。

请注意：与上个学期以及寒假考核的合格制不同，本学期的日常考核以及暑假考核均采用末位淘汰制，请大家务必重视自己项目的质量，并尽可能多地完成下面的考核任务。

二、学习内容和考核任务

1. Experience Products

概述

要进行 Agent 相关的产品开发，我们必须先了解一个好的、能够被广泛应用于现实场景的 Agent 产品是什么样的，因此就有必要广泛地了解和体验各种 Agent 产品。

学习目标

亲自上手体验 Agent 产品，对此建立基础认知，并思考、学习其思路和实现方式。

学习内容和考核任务

1. 体验至少两种基于 LLM Agent 的编程工具（推荐每种类型的工具都尝试一下），例如编辑器类的 [Cursor](#)、[WindSurf](#)、[Kiro](#)、[Antigravity](#)、[Trae](#)、[CodeBuddy \(IDE\)](#)、[通义灵码 \(IDE\)](#)，编辑器扩展类的 [GitHub Copilot in VS Code](#)、[Cline](#)、[Roo Code](#)、[BlackBox AI](#)、[CodeBuddy \(扩展\)](#)、[通义灵码 \(扩展\)](#)、等，以及命令行类的 [Claude Code](#)、[Codex CLI](#)、[Gemini CLI](#)、[OpenCode](#) 等。
2. 体验至少两种 Agent Workflow & Knowledgebase 平台，例如 [Dify](#)、[n8n](#)、[RAGFlow](#)、[DeerFlow](#)、[Coze Studio](#)、[AnythingLLM](#) 等。
3. 体验至少两种 Claw 类工具，例如 [OpenClaw](#)、[ZeroClaw](#)、[OpenFang](#)、[Eigent](#) 等，以及由厂商开发的 Claw 类工具，如 [MaxClaw](#)、[Kimi Claw](#)、[CoPaw](#)、[QClaw](#)、[360安全龙虾](#) 等。
4. （选做）体验 [ComfyUI](#)。

以上内容均要求在体验的同时对其思路和实现原理进行思考，并通过各种工具对相关信息进行检索和验证。

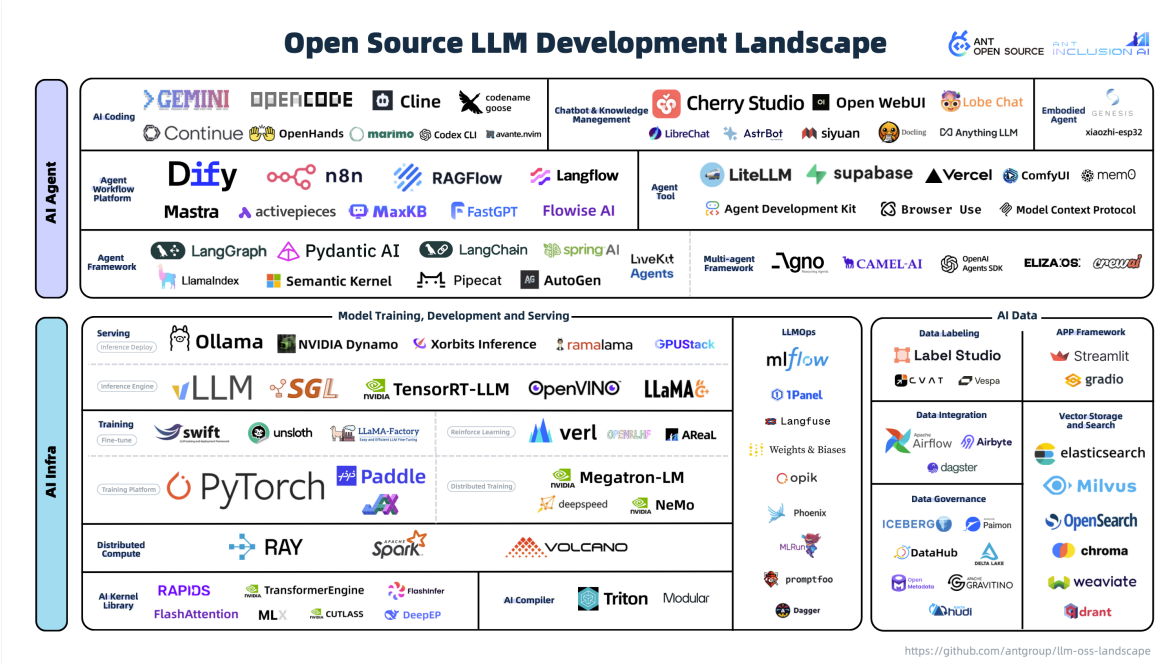
提交内容

一份 PDF 文档，其中需要以图文的方式记录你了解和体验了哪些 Agent 产品、体验的过程，还有你对其实现原理的思考和复现过程。

2. Master Stacks

概述

要实现一个生产级的 Agent 系统，开源的框架、组件等基础设施/技术栈通常是必不可少的。下图为蚂蚁开源在 2025 年 9 月发布的开源 LLM 开发生态全景图，可作为学习和项目技术选型的参考。



学习目标

学习和掌握 Agent 开发常用的基础设施，如 Agent 编排框架、LLM API 网关、向量数据库等。

学习内容和考核任务

- 学习并掌握至少一种 Agent 框架，例如 [LangChain](#)、[LangGraph](#)、[Pydantic AI](#)、[AutoGen](#)、[LiveKit Agents](#) 等，并围绕你学习的 Agent 框架开发一个项目，或将其集成到你已有的项目中。
- 学习并掌握至少一种 LLM API 网关，例如 [LiteLLM](#)、[Portkey AI Gateway](#)、[OneAPI](#) 等，并围绕你学习的 LLM API 网关开发一个项目，或将其集成到你已有的项目中。
- 学习并掌握至少一种向量数据库，例如 [Chroma](#)、[Milvus](#)、[Weaviate](#)、[Qdrant](#) 等，并围绕你学习的向量数据库开发一个项目，或将其集成到你已有的项目中。
- （选做）学习并掌握至少一种图数据库，例如 [neo4j](#)、[SurrealDB](#) 等，并围绕你学习的图数据库开发一个项目，或将其集成到你已有的项目中。
- （选做）学习并掌握至少一种文档解析和转换器，例如 [MinerU](#)、[Unstructured](#)、[MarkItDown](#)、[Marker](#)、[Docling](#) 等，并围绕你学习的文档解析和转换器开发一个项目，或将其集成到你已有的项目中。

对于以上任务，你既可以为每一个任务编写一个单独的项目，也可以选择在一个项目中（不管是新的还是已有的）集成上面所有内容。

提交内容

一份 PDF 文档，包含可访问的代码仓库地址、你所开发的项目介绍，以及你进行项目开发/完善过程的记录。

3. Further RAG

概述

在上个学期以及寒假中，我们已经通过自学的方式对 RAG（Retrieval-Augmented Generation，检索增强生成）技术及其流程和实现有了基本的了解，但大家所开发的 RAG 项目仍然非常粗糙，属于最简单的 Naive RAG，在检索和生成效果上都不尽如人意。因此，我们仍需对 RAG 技术进行更进一步的学习，并通过项目开发的方式对所学进行实践。

学习目标

本次任务要求大家深入学习 Advanced RAG、Modular RAG、Hybrid RAG、Graph RAG 以及 Agentic RAG 等高级 RAG 技术，并结合之前学习过的向量数据库、图数据库、文档解析器等工具，构建一个 Agentic RAG 项目。

学习内容和考核任务

在自学相关 RAG 原理、技术和工具后，开发一个 RAG 项目或在已有的 RAG 项目上进行修改和完善（对于已经做过 RAG 项目的同学）。

这里的 RAG 项目不拘泥于传统的文档知识库，不管是作为 LLM 驱动的搜索引擎的一部分，还是作为代码仓库的深度理解能力集成到 Agentic Coding/Wiki 项目中，都是完全可以的，但**必须用到 Agentic RAG 方法，并至少使用三种进阶 RAG 技术**。

进阶 RAG 技术有很多，包括但不限于下面这些：

- 1. Hierarchical Index Retrieval（层次索引）**：对于大型数据源，在进行 RAG 处理时可以创建多级索引。例如，我们在程序中为一个来自文档的数据源创建两层索引，一层由摘要组成，另一层由文档块组成，然后分为两步进行搜索：首先通过摘要过滤出相关文档，接着只在这个相关文档内进行搜索。层次索引能够提高检索的效率和准确性。
- 2. HyDE（Hypothetical Document Embeddings，假设性文档嵌入）**：在处理用于 RAG 的数据源时，我们可以让 LLM 为每个块生成一个假设性问题，并将这些问题以向量形式嵌入。在检索时，针对这个问题向量的索引进行查询搜索（即用问题向量替换索引中的块向量），检索后将对应的原始文本块作为上下文发送给 LLM 以获取答案。因为查询和假设性问题之间的语义相似性更高，这样做通常能显著提高搜索质量。我们还可以将 HyDE 方法反过来用，也就是让 LLM 根据查询内容先生成一个假设性回答，然后将该回答的向量与查询向量一起进行检索，也可以提高检索和生成的质量。
- 3. Context Enrichment（上下文增强）**：在检索时仍然检索较小的块以提高检索性能和质量，但在生成时增加周围的上下文，以供 LLM 进行推理分析。可以在检索到的较小块周围添加句子来扩展上下文，也可以递归地将文档分割成多个包含较小子块的大型父块，然后根据一定的规则对父块进行匹配，然后在生成时匹配到的父块（而不只是检索到的子块）发送给 LLM。
- 4. Graph RAG（基于图的 RAG）**：如果用户的问题涉及大量散落在数据源中的信息，或需要通过逻辑推理对查询结果进行处理，那么传统的、仅基于关键词匹配或向量检索的 RAG 方法效果可能会很差。为此，我们可以引入知识图谱以及图数据库，在原始文本切块处理之后，使用 LLM 提取原始文本中的实体和关系，再将所有文本块中提取的实体和关系汇总，经过去重、合并，就构成了一张完整的知识图谱。将构建好的知识图谱存入图数据库后，就可以通过 Leiden 等图聚类算法在知识图谱上进行分层和摘要处理，最后在检索时就可以通过摘要快速定位到知识图谱中的对应实体节点，并根据节点之间的关系链路进行探索式检索。
- 5. Hybrid Search（融合检索）**：使用多种不同的方法进行检索，例如经典的关键词匹配和向量检索，以兼顾搜索的性能和准确性，然后按照一定的规则对不同方法检索到的结果进行评分、重排和筛选。

6. **Query Transformation (查询转换)**：使用 LLM 将用户的查询内容重写为多个子查询，然后并行执行这些查询，最后将这些子查询的检索结果进行汇总和处理后，再发送给 LLM 进行生成。
7. **Context Compression (上下文压缩)**：用户通常会在一次聊天中与 LLM 进行多轮对话，不同轮次的对话可能涉及到之前已有的 RAG 查询结果，也可能与之前已有的 RAG 查询结果毫不相干，或是需要进行新的 RAG 查询。因此，我们需要引入一种机制，在多轮对话中对用户的每一次提问进行上下文的压缩和重新组织，这样既能减小 Token 开销，又能提高 LLM 的回复质量。
8. **Multi-Agent (多智能体)**：在一个生产级的 RAG 项目中，我们可能需要对多个不同的数据源进行查询，而这些数据源又可能需要使用不同的查询方法。例如，对文档形式的知识库，我们可以使用关键词检索和向量检索，而对一个 Web 后端服务的业务数据库来说，直接使用 SQL 语句进行检索效果可能会更好。因此，我们需要将 RAG 的流程解耦，并通过多个使用不同 RAG 流程的 Agent，分别适配不同数据源和不同方法的检索。
9. **Query Router (查询路由)**：查询路由是一种基于 LLM 的决策方法，用于确定针对用户的查询接下来应采取的行动，例如调用子 Agent 针对某个数据索引执行搜索（即引导到子链或其他智能体），然后将结果综合成一个答案。

提交内容

一份 PDF 文档，包含可访问的代码仓库地址、你所开发的项目介绍，以及你进行项目开发/完善过程的记录。

4. Autonomous Agents

概述

经过前面的学习，我们已经了解了基于 Agent 的产品是如何运作的，也掌握了构建 Agent 所需的基础工具，但我们之前所编写的诸多 Agent 项目都缺少任务规划、调用工具以及执行命令的能力，不能够很好地完成真实世界中的任务。为此，我们需要进行更深入的学习和开发。

学习目标

学习和掌握如何构建一个能够自主进行任务拆解和规划、调用工具以及执行命令的通用 Agent，并进行实践。

学习内容和考核任务

学习 [Deep Agents](#)、[OpenAI Agents SDK](#) 等项目和工具，在你现有的项目或新的项目中实现通用自主 Agent，要求至少能够进行任务规划、文件读取/编辑以及命令执行，并能够在任务上持续工作。对于文件读写以及命令执行功能，**需要实现相应的安全机制**，例如限制读写目录范围、命令手动/自动审批、在独立的沙箱中执行命令等。

提交内容

一份 PDF 文档，包含可访问的代码仓库地址、你所开发的项目介绍，以及你进行项目开发/完善过程的记录。

5. Agent Skills

概述

当前，AI Agents 已经从概念实验迅速走向了工程实践，不再满足于“能回答问题”，而是要真正执行任务、嵌入业务流程、扩展工具能力并自动化工作。在这样的背景下，对于特定领域的一些任务，一个核心构建单元开始显得至关重要——Agent Skills（智能体技能）。

学习目标

掌握编写 Agent Skills 以及在项目中实现 Agent Skills 使用和管理能力（Client）的方法，并进行实践。

学习内容和考核任务

参考 [Agent Skills 官方文档](#) 进行学习，针对任意一个领域编写一份 Agent Skill 并进行调试，然后为你的项目引入使用和管理 Agent Skill 的能力，或开发一个新的项目实现 Agent Skill 的使用和管理。

提交内容

1. 一份 PDF 文档，包含编写和调试 Agent Skill 的记录、你项目的可访问的代码仓库地址、你所开发的项目介绍，以及你进行项目开发/完善过程的记录。
2. 一个 ZIP/RAR/7Z 压缩文件，包含你所编写的完整 Agent Skill。

6. Agent Memory

概述

在此前的学习中，我们了解并实践了 Agent 的基本架构、开发框架以及工具调用能力。然而，要实现通用人工智能（Artificial General Intelligence, AGI）的终极目标，Agent 必须学会自己探索世界、从经验中学习和不断进化——就像人一样。而要做到这一点，仅靠上下文窗口是远远不够的。这时候，就需要 Agent Memory（智能体记忆）登场了。

学习目标

了解 Agent Memory 的基础概念，掌握其实现方式以及常见的 Agent Memory 框架，并进行实践。

学习内容和考核任务

学习 [mem0](#)、[MemOS](#)、[Letta](#)、[Memori](#)、[Honcho](#)、[LangMem](#) 等项目和框架，并将其接入到你现有的项目中，或开发一个新的项目。

提交内容

一份 PDF 文档，包含可访问的代码仓库地址、你所开发的项目介绍，以及你进行项目开发/完善过程的记录。

7. GUI Agents

概述

在过去二十多年的时间里，自动化的主流方案是 RPA（机器人流程自动化）。然而，RPA 有一个致命弱点：依赖于固定的 UI 元素选择器（Selectors）。一旦界面稍有变化，先前的 RPA 脚本就会失效。这种脆弱性导致了巨大的维护成本，也限制了其应用领域。

时过境迁，GUI Agent 的出现彻底改变了这个局面。区别于简单地回放预设的脚本，GUI Agent 能够像人类一样，通过视觉动态感知理解屏幕内容，并通过 LLM 的推理能力规划操作路径，从而在动态、未知的软件环境中自主完成任务。

学习目标

1. 学习和掌握让 Agent 获取及操作 Web 页面和浏览器本身的方法，并进行实践。
2. 学习和掌握让 Agent 进行桌面/移动端 GUI 操作的方法，并进行实践。
3. 了解 OSWorld、AndroidWorld 等 GUI Agents 榜单及评测方法。

学习内容和考核任务

1. 学习 [Firecrawl](#)、[Browser Use](#)、[Page Agent](#)、[Nanobrowser](#) 等项目，并将其接入到你现有的项目中，或开发一个新的项目。
2. 学习 [Midscene.js](#)、[Agent S](#)、[cua](#)、[Open-AutoGLM](#)、[TuriX-CUA](#)、[UI-TARS](#)、[Mobile-use](#) 等项目，并将其接入到你现有的项目中，或开发一个新的项目。
3. 通过 [OSWorld](#)、[AndroidWorld](#) 等公开 GUI Agents 评测基准对你所开发的 GUI Agents 进行评测，并记录过程。

提交内容

一份 PDF 文档，包含可访问的代码仓库地址、你所开发的项目介绍，以及你进行项目开发/完善和 GUI Agents 评测过程的记录。

8. Multi-Agent System

概述

在过去相当漫长的时间里，人类的单体智能并未有显著的增长，但人类社会的整体能力却在信息时代发生了指数级提升。这一方面是因为计算机等强大工具的出现，另一方面也是人类协作与群体智慧的结果。

这一规律也适用于 LLM。随着单个 LLM 的能力增长逐渐放缓，Multi-Agent System（多智能体代理系统）正在成为新的研究热点。

学习目标

了解多智能体系统相关的概念，学习和掌握构建自主多智能体系统的方法，并进行实践。

学习内容和考核任务

学习 [Agno](#)、[CAMEL](#)、[CrewAI](#)、[elizaOS](#) 等项目和框架，并在你现有的项目或新的项目中实现自主多智能体系统。

提交内容

一份 PDF 文档，包含可访问的代码仓库地址、你所开发的项目介绍，以及你进行项目开发/完善过程的记录。

三、项目开发要求

对于本学期日常任务中需要进行项目开发的部分，你所开发的项目必须满足如下要求：

1. 如果你的程序使用 Python 编写，或包含使用 Python 编写的部分，那么必须使用 uv 进行 Python 项目管理。

2. 除了作为通用默认值（常见于模型名称和推理服务提供商的 API 端点）的情形外，不得在程序代码中硬编码配置信息（尤其是 API Key），必须支持在部署或使用由用户修改这些配置信息。可以使用环境变量（包括 .env 文件）或独立的配置信息文件来实现，也可以在项目中集成 LiteLLM、OneAPI 等 LLM API 聚合框架，以此支持多个 LLM API 供应商。
3. 项目必须具有用户界面，无论是 WebUI、GUI 还是 TUI，甚至 CLI 都是可以的，但不要出现仅有 Web 后端而没有相应前端界面的情况。
4. 必须使用 Git 管理你的项目，并将你的项目代码提交和推送到任意一个公共代码托管平台上。
5. 对于 Web 全栈项目或项目中的 Web 前端和后端部分，需要编写 Dockerfile 将其容器化，并在提交的文档中记录构建容器镜像以及运行容器镜像的过程。

四、考核提交方式与截止时间

本学期的日常考核提交方式与上学期相同，仍然是在红岩网校作业提交平台上进行线上提交。

提交格式要求为单个 PDF 文档，PDF 文档的文件大小限制为 30MB。对于考核任务中项目开发的部分，需要使用 Git 将你的代码提交并推送到任意公共代码托管平台（如 GitHub、CodeBerg、Gitee 等）上，并在提交的 PDF 中附上可访问的代码仓库链接。

日常考核提交入口自本文档发布之日起开放，到本学期十八周周日（2026年7月5日）23:59 截止，在此期间均可提交。对同一次考核任务重复提交的，以最后一次提交的内容为准。